

SERVICE ACTUEL · DATA-COLLECT

Connexions, dépôts et collectes de datasets

27 capacités analysées

services/data-collect · Kotlin 2.4.10 et Spring Boot 4.1.1 observés dans le catalogue Gradle ; modules domaine et adaptateurs Spring ; PostgreSQL/JPA/Flyway, Quartz JDBC, Kafka/Protobuf, SSH, client S3, JDBC et Parquet/Hadoop sur JVM.

Le service administre les sources et dépôts, aide à choisir et vérifier une lecture, collecte les octets dans le stockage d'import, planifie les reprises et échange les états avec data-dev, data-model et process-data. Il sert aussi les fichiers destinés aux corpus IA et le catalogue de sources du SDK Python.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Connectors, Datasets, Flows, Writer, Publisher, Files, IAM, Audit, Egress, Maintenance, Notifications, Embeddings.

Périmètre de lecture

- AGENTS.md du service et règle Helm commune.
- Tous les contrôleurs REST : datasource, fichiers, ingestion IA, permission, historique, KPI, config dataset, propriétaires, SFTP et administration.
- Registres et variantes de DTO de connexion/configuration/format ; agrégats datasource, dataset, permission, fichiers et schedules.
- Handlers de création, modification, suppression, partage, preview, contrôle de schéma, import, refresh, finalisation, dépôt et transfert de propriété.
- Factory et managers JDBC/API/S3/SFTP/local, stratégies API de pagination/découverte/authentification, stockage S3, cache de listing, chiffrement et reprise.
- Consumers data-model/process-data/Depot/IAM, droits résiduels, scheduler et démarrage ; chart Deployment, NetworkPolicy et HPA.
- Inventaire des tests de contrats concernés ; aucun build, test applicatif, cluster ou connecteur distant exécuté.

Objets et cycle de vie

- DataSource identifie une connexion et son propriétaire. La configuration de lecture du dataset est séparée de la connexion et peut figer une version API.
- ResourceDescriptor se spécialise en FileDescriptor ou TableDescriptor. Les identifiants sont réconciliés à la redécouverte ; les propriétés de navigation ne constituent pas un snapshot garanti de la source distante.
- DataSourceProjectPermission conserve demande, décision, justification et acteurs ; la projection IAM matérialise le droit d'usage accepté.
- DatasetCollect conserve nom, propriétaire, projet optionnel, configuration courante/ancienne et refreshConfig. CollectScheduled encode une étape asynchrone et son budget de reprise.
- SftpAccess et SftpAccessAuthorization reflètent un compte distant et ses clés publiques ; SftpExpireTime décrit une rétention distincte de l'expiration d'accès.

- FolderListingGeneration et ses entrées forment un cache jetable ; historique, UsageRecord, maps d'identité, files différées et outbox ne doivent pas être confondus avec les checkpoints de données publiées.

Dépendances du service

- Data-dev produit les créations de collectes et les anciens miroirs projet/utilisateur. Data-model produit changements de dataset, de schéma, de schedule et vidages.
- Process-data retourne les événements de fin d'exécution. data-collect publie l'état ingestion/process et la demande de suite de traitement par Kafka/Protobuf.
- Notification-center reçoit les échecs de collecte, demandes/décisions de permission, abonnements automatiques et événements synchrones de contrôle de fichier.
- IAM fournit visibilité, autorisations déléguées et événements de cycle de vie. APISIX contrôle les routes exposées ; IAM écrit OpenFGA à partir des faits data-collect.
- Les octets transitent par connecteurs externes, cache local et bucket S3 d'import. Le SDK Python consomme la liste autorisée de sources. ai-gen-assistant consomme listing et contenu sous délégation.

Fonctionnalités et compatibilité à préserver

DC-01 Cible non qualifiée

Enregistrer et faire évoluer une datasource

COMPORTEMENT OBSERVÉ

Une datasource porte un UUID, un nom, une description, un propriétaire, une sensibilité, un connecteur, un état et un historique. Le DTO de création permet un propriétaire explicite et applique ses propres valeurs par défaut. Le manager du connecteur construit les descripteurs initiaux. La modification conserve les secrets laissés vides et interdit de changer de type de connecteur. La suppression retire les liens projet et les configurations de collecte, demande la suppression des imports, puis émet les événements de suppression. La création publie un fait de propriété et attend sa visibilité dans IAM. Les erreurs métier ne suivent pas toutes la même convention HTTP, certains handlers renvoient null puis 400.

UTILISATEURS ET RÔLES

Gestionnaires de sources, propriétaires, administrateurs et créations automatiques depuis un dépôt.

INTERFACES DANS LE CODE

POST /datasources ; GET /datasources/{id} ; PUT /datasources/{id} ; DELETE /datasources/{id}.

DONNÉES ET ÉTATS

DataSource, Connector, ResourceDescriptor, DataSourceHistory et UsageRecord en PostgreSQL. SUCCESS, WARNING et FAILURE décrivent la source.

DÉPENDANCES ET ÉCHANGES

ConnectorFactory ; stockage objet des imports ; événements DataSourceCreateEvent, DataSourceUpdateEvent, DataSourceDeleteEvent ; IAM pour la visibilité du propriétaire.

COMPORTEMENT CIBLE PROPOSÉ

Connectors devient propriétaire des définitions de connexion en Rust. Datasets référence leur UUID et une révision immuable de configuration. Toute opération possède un identifiant idempotent, un acteur et un tenant.

MIGRATION DE CETTE CAPACITÉ

Importer les UUID et les références existantes. Séparer la désactivation de source de la suppression physique des données. Fournir un adaptateur de routes et expliciter les erreurs de création et de modification.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer une source, modifier sa description sans renvoyer son secret, puis relire ses capacités. Une modification de type est refusée. Supprimer une source liée produit un état de retrait traçable sans supprimer une édition publiée.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:190](#) Création, visibilité IAM et routes de modification/suppression.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/DataSource.kt:101](#) Immutabilité du type et calcul des changements.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/DeleteDataSourceCommandHandler.kt:42](#) Effets de la suppression sur liens, collectes et imports.

DC-02 **Cible non qualifiée****Chercher des sources accessibles et les exposer à la bibliothèque Python****COMPORTEMENT OBSERVÉ**

La liste combine recherche textuelle, tri, pagination, projet, types de connecteurs, sensibilités et statuts des demandes. Elle récupère les UUID candidats puis interroge IAM avant la pagination visible. Le mode `notRequested` suit un parcours distinct où `visibleIds` reste null. Les UUID de projet entrants sont traduits vers les anciennes clés de stockage. La bibliothèque Python dispose d'une route propre qui ne retient que les sources acceptées dans le projet et applique la même visibilité IAM. Une ancienne variante sélectionnée par User-Agent `pylib` subsiste et est dépréciée.

UTILISATEURS ET RÔLES

Utilisateurs de catalogue, membres de projets, notebooks et bibliothèque Python.

INTERFACES DANS LE CODE

GET `/datasources` ; GET `/lib/datasources?projectId=...` ; ancienne variante GET `/datasources` sélectionnée par User-Agent.

DONNÉES ET ÉTATS

Permissions projet, miroirs de projets, correspondances IAM/legacy et pagination calculée sur les candidats autorisés.

DÉPENDANCES ET ÉCHANGES

`IamDataSourceVisibilityClient` et `ProjectIdMap` ; contrats consommés par les frontends et les bibliothèques.

COMPORTEMENT CIBLE PROPOSÉ

Catalog et Connectors servent une recherche filtrée par IAM avec pagination stable. Le SDK Python consomme les mêmes identifiants canoniques et ne reçoit aucune capacité de connexion permanente.

MIGRATION DE CETTE CAPACITÉ

Conserver les filtres et formes de réponse du SDK pendant la transition. Retirer la sélection par User-Agent après migration des clients. Définir le contrat exact de `notRequested` avant sa reprise.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membre voit seulement les sources autorisées sur chaque page et dans le SDK. Les totaux reflètent ce filtrage. Un même projet envoyé sous son ancien ou son nouvel UUID ne crée pas deux partitions de droits.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:90](#) Filtres et vérification des candidats par IAM.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceForLibController.kt:45](#) Route SDK, sources acceptées et dépréciation.

DC-03 **Cible non qualifiée**

Demander et décider un accès projet à une source**COMPORTEMENT OBSERVÉ**

Un membre demande l'activation d'une source dans un projet avec un commentaire. Le handler réutilise une demande déjà acceptée ou en attente ; une demande refusée permet une nouvelle demande. Il stocke le demandeur et le projet, émet un événement de permission, notifie la demande et abonne le demandeur aux réponses INAPP. Le décideur ne peut fixer que ACCEPTED ou REFUSED et ajoute justification, date et validateur. L'acceptation publie un fait de lien, le refus un fait de révocation. Une route liste les demandes filtrées et paginées.

UTILISATEURS ET RÔLES

Membres de projet, responsables de sources et administrateurs selon les routes IAM.

INTERFACES DANS LE CODE

POST /datasources/{id}/projects/{projectId} ; POST /datasources/permissions/{permissionId}/{permission} ; GET /datasources/permissions.

DONNÉES ET ÉTATS

DataSourceProjectPermission : REQUESTED, ACCEPTED ou REFUSED ; auteur, commentaire, validateur, justification et horodatages.

DÉPENDANCES ET ÉCHANGES

Notifications de demande/réponse ; abonnement automatique ; IAM reçoit les faits de lien via outbox.

COMPORTEMENT CIBLE PROPOSÉ

IAM tranche les droits, Connectors garde le workflow de demande et Audit son historique. Le lien approuvé donne un droit d'usage, sans exposer les credentials au projet.

MIGRATION DE CETTE CAPACITÉ

Reprendre les demandes actives et les décisions passées en conservant les identifiants. Dédupliquer par source/projet/demande et reconstruire les faits IAM avant bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux envois d'une demande active retournent le même dossier. Un refus motivé laisse une trace et permet une nouvelle demande. Une acceptation devient visible dans le SDK puis une révocation retire l'accès.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceLinkToProjectCommandHandler.kt:43](#) Création, notifications et abonnements de réponse.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/AcceptDataSourceLinkToProjectCommandHandler.kt:43 Décision et faits IAM.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourcePermissionController.kt:65 Routes et traduction des UUID projet.

DC-04 Cible non qualifiée

Auditer les changements et compter les sources et propriétaires

COMPORTEMENT OBSERVÉ

L'historique présente les créations et modifications, de la plus récente à la plus ancienne. Les changements conservent les valeurs avant/après des champs non secrets ; un changement de secret conserve seulement le nom du champ concerné. La recherche peut porter sur les champs modifiés et leurs valeurs. Les routes KPI de source et de dépôt renvoient des comptages. Le contrôle de propriétaire consulte séparément les sources et les datasets publics pour préparer les transferts de propriété.

UTILISATEURS ET RÔLES

Propriétaires, administrateurs, parcours de départ utilisateur et support.

INTERFACES DANS LE CODE

GET /datasources/{id}/history ; GET /kpi/datasources ; GET /sftp-accesses/kpi ; GET /users/{userId}/check-owner.

DONNÉES ET ÉTATS

DataSourceHistory et DataSourceFieldChange ; journaux UsageRecord ; relations de propriété.

DÉPENDANCES ET ÉCHANGES

Repositories de sources, collectes et dépôts ; miroirs des auteurs.

COMPORTEMENT CIBLE PROPOSÉ

Audit conserve des événements signés logiquement par acteur, commande et révision. Catalog fournit les compteurs avec une définition de périmètre tenant. IAM utilise un inventaire de propriété pour les départs.

MIGRATION DE CETTE CAPACITÉ

Convertir les historiques existants sans réintroduire les anciennes valeurs secrètes. Versionner les noms de champs recherchables et conserver les références d'auteur même après désactivation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une rotation de credential apparaît dans l'historique sans valeur avant/après. Une recherche sur connector.host retrouve les modifications attendues. Les compteurs restent limités au tenant demandé.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/DataSource.kt:135 Diff des champs ordinaires et SecretChange sans contenu.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceHistoryController.kt:21 Filtres de l'historique.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/UserController.kt:21 Contrôle des propriétés de source et dataset public.

DC-05 **Cible non qualifiée**

Tester une connexion et réutiliser un secret masqué

COMPORTEMENT OBSERVÉ

Le test d'une nouvelle connexion reçoit un ConnectorDTO. Le test d'une source persistée utilise une route portant l'UUID ; le paramètre historique datasourced sur la route générale est refusé en 422. Pour une source existante, les secrets vides sont remplacés par les secrets stockés, puis le manager vérifie la connexion. Le résultat expose un booléen et une catégorie d'erreur, sans renvoyer le message brut d'exception. Les DTO sortants passent par maskSecrets. Les secrets vides lors d'une modification ont aussi une sémantique de conservation.

UTILISATEURS ET RÔLES

Créateurs et gestionnaires de connexions.

INTERFACES DANS LE CODE

POST /datasources/check ; POST /datasources/{id}/check ; Connector.toDTO ; Connector.withStoredSecrets.

DONNÉES ET ÉTATS

Configuration soumise et éventuelle configuration persistée ; absence de mutation de la définition requise pour un simple test.

DÉPENDANCES ET ÉCHANGES

Managers JDBC, SFTP, S3 et HTTP ; catalogue ConnectionErrorType ; credentials déchiffrés côté service.

COMPORTEMENT CIBLE PROPOSÉ

Connectors exécute un test borné avec la révision et la référence de secret autorisées. Egress décide les destinations et Audit consigne la catégorie d'échec sans credential.

MIGRATION DE CETTE CAPACITÉ

Conserver le contrat de masquage et le comportement des champs vides. Remplacer ce mécanisme ambigu à terme par une commande explicite preserve/replace/clear.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un test sur une source masquée réussit avec son secret stocké ; un nouvel endpoint arbitraire ne peut pas réutiliser un secret non autorisé. Un refus distant retourne une catégorie stable sans texte confidentiel.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceController.kt:155](#) Deux routes de test et rejet du paramètre historique.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DataSourceCheckQueryHandler.kt:38](#) Fusion des secrets et catégories d'erreur.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/dto/connector/ConnectorDTO.kt:107](#) Masquage systématique des DTO sortants.

DC-06 **Cible non qualifiée**

Chiffrer et faire tourner les credentials de connexion

COMPORTEMENT OBSERVÉ

Les nouveaux credentials utilisent une enveloppe AES-GCM versionnée avec identifiant de génération, nonce aléatoire et données associées. Le décodeur sait encore lire l'ancien format CBC. Une génération précédente peut coexister avec l'actuelle ; la sélection de clé legacy devient explicite en présence des deux. La rotation vérifie même une enveloppe déjà courante. Le ré-encrypteur parcourt les champs de secrets et les entrées API headers/params/body dans le JSON persistant. Une donnée illisible lève une erreur contextualisée par chemin, sans la convertir en chaîne vide.

UTILISATEURS ET RÔLES

Opérateurs de secrets et migration de connexions.

INTERFACES DANS LE CODE

AttributeEncryptor.encoder/decoder/rotate ; DatasourceCredentialReEncryptor.reEncrypt ; runner de ré-encryption.

DONNÉES ET ÉTATS

Payload JSON datasource chiffré, générations actuelle et précédente ; l'enveloppe porte la version et la clé logique.

DÉPENDANCES ET ÉCHANGES

Secrets de déploiement ; partage historique de la clé avec data-dev signalé dans AGENTS.md.

COMPORTEMENT CIBLE PROPOSÉ

Connectors et IAM utilisent des références de secrets. Un module Rust de compatibilité déchiffre les enveloppes existantes, puis les migre vers un stockage de secrets avec génération, droits et journaux de rotation.

MIGRATION DE CETTE CAPACITÉ

Lire le guide de rotation avant toute opération. Migrer par lots vérifiables et ré-exécutables, conserver l'ancienne génération jusqu'à preuve que tous les lecteurs ont basculé, puis révoquer.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une enveloppe de chaque génération connue reste lisible pendant la transition. Une altération est rejetée. Relancer un lot ne modifie pas une enveloppe déjà validée et aucun journal ne contient le secret.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/AttributeEncryptor.kt:65](#) Formats de lecture et rotation authentifiée.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/AttributeEncryptor.kt:128](#) Nonce, chiffrement GCM et données associées.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/utils/DatasourceCredentialReEncryptor.kt:41](#) Réécriture du payload et champs couverts.

DC-07 **Cible non qualifiée**

Découvrir et extraire les tables SQL

COMPORTEMENT OBSERVÉ

Le manager JDBC découvre schémas, tables et vues, normalise les types de table et conserve les UUID de descripteurs lors d'une redécouverte. Il applique des règles propres à PostgreSQL, MySQL, SQL Server, Oracle et Snowflake, y compris certaines vues matérialisées ou tables externes. Le preview lit un nombre limité de lignes. L'import lit un flux JDBC, écrit un fichier Parquet local compressé puis l'envoie au bucket d'import. Les comptes de lignes et colonnes sont recalculés séparément. BIGQUERY est déclaré mais sa sélection de driver est vide dans le code audité ; son fonctionnement ne peut pas être affirmé.

UTILISATEURS ET RÔLES

Analystes qui importent des tables, gestionnaires de connexions SQL et workers de collecte.

INTERFACES DANS LE CODE

GET /datasources/{id}/resources ; GET /datasources/{id}/datasets/query ;
JdbcConnectorManager.list/read/import.

DONNÉES ET ÉTATS

TableDescriptor avec table, schéma, type et identifiant stable ; schémas inférés ; Parquet temporaire et objet d'import.

DÉPENDANCES ET ÉCHANGES

Drivers JDBC et bibliothèques Parquet/Hadoop JVM ; bases externes ; stockage compatible S3.

COMPORTEMENT CIBLE PROPOSÉ

Connectors fournit des lecteurs Rust ou des tiers Go/C++ encapsulés et qualifiés par moteur. Writer transforme des lots bornés en fichiers immuables, sans droit REST Iceberg générique. Publisher publie les éditions TABLE/INCREMENTAL.

MIGRATION DE CETTE CAPACITÉ

Construire une matrice de qualification par moteur, type de colonne, pagination et stratégie snapshot. Remplacer les drivers JVM un par un ; ne pas déclarer BIGQUERY pris en charge sans un adaptateur exécutable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Pour chaque moteur retenu, importer une table avec décimaux, valeurs nulles, fuseaux et noms de colonnes particuliers. Comparer lignes, types et valeurs à une extraction de référence et vérifier une reprise sans doublon.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:103](#)

Découverte et conservation des descripteurs.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:630](#) Flux

JDBC, Parquet local puis S3.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:736](#)

Drivers associés aux types dont BIGQUERY vide.

DC-08 Cible non qualifiée

Valider une requête SQL personnalisée sur une table

COMPORTEMENT OBSERVÉ

Une configuration SQL peut fournir customQuery. Le validateur parse exactement une instruction SELECT simple. Il refuse les JOIN, les CTE et les sous-requêtes et exige que la table, et le schéma quand il est connu, correspondent au descripteur sélectionné. Le preview et la création appellent cette validation. Les colonnes peuvent être inférées à partir des métadonnées du résultat, puis l'import utilise la requête. Ce contrat n'est donc pas un moteur SQL général ni une garantie de lecture sans effet propre au compte distant.

UTILISATEURS ET RÔLES

Analystes et auteurs de configurations de collecte SQL.

INTERFACES DANS LE CODE

Champ customQuery des configurations SQL ; routes datasets/preview et datasets/headers ; CustomQueryValidator.validate.

DONNÉES ET ÉTATS

Requête rattachée au dataset et schéma calculé sur son résultat, avec descripteur de la table source.

DÉPENDANCES ET ÉCHANGES

JSQLParser, métadonnées JDBC et compte distant.

COMPORTEMENT CIBLE PROPOSÉ

Connectors possède un sous-langage de lecture explicite et validé. Les transformations SQL générales appartiennent à Flows/Query. Le compte externe est en lecture seule et Egress borne la durée et les volumes.

MIGRATION DE CETTE CAPACITÉ

Versionner la grammaire et reprendre les requêtes acceptées sans élargir silencieusement leurs droits. Prévoir une migration explicite vers Flows pour les demandes de jointure.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une projection filtrée de la table sélectionnée est acceptée. Une seconde instruction, un JOIN, une CTE ou une table différente est refusé avant toute exécution distante.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/CustomQueryValidatorImpl.kt:21 Validation syntaxique et table cible.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/PreviewDatasetConfigQueryHandler.kt:115 Utilisation du validateur dans le preview.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:287 Inférence du schéma de customQuery.

DC-09 **Cible non qualifiée**

Appeler des API avec authentification et reprises HTTP

COMPOTEMENT OBSERVÉ

Le connecteur API combine URL de base, endpoint, paramètres, en-têtes et méthode. L'authentification peut être absente, Basic ou fondée sur un endpoint de jeton. Ce dernier accepte GET/POST et injecte le token dans un header ou un paramètre selon un motif ; le cache tient compte d'une durée de vie fournie. Le transport distingue refus d'identité, refus d'accès, absence d'endpoint, timeout et autres statuts. Il réessaie selon un budget de temps et une politique de backoff, interprète Retry-After et peut invalider un token sur 401. Les téléchargements consomment un flux et suppriment un fichier partiel lors d'un échec.

UTILISATEURS ET RÔLES

Gestionnaires de connexions HTTP et collectes de données SaaS.

INTERFACES DANS LE CODE

APIConnectorManager ; ApiAuthenticator ; HttpGateway.exchange/download ; routes de test et preview.

DONNÉES ET ÉTATS

Définition de connexion, configuration par dataset, cache de jetons et fichier temporaire pour les octets.

DÉPENDANCES ET ÉCHANGES

API distante, endpoint de token et client HTTP mutualisé.

COMPOTEMENT CIBLE PROPOSÉ

Connectors implémente les mêmes modes avec références de secrets, requêtes bornées et catégories d'erreur stables. Egress autorise URL, redirections et hôtes de token avant chaque connexion.

MIGRATION DE CETTE CAPACITÉ

Conserver les options effectivement utilisées et établir des tests de contrat par méthode et fournisseur. Séparer les paramètres secrets des paramètres de données pour le masquage et l'audit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une réponse 401 provoque au plus le renouvellement prévu, un 429 respecte son délai dans le budget, et un flux interrompu ne laisse aucun fichier réutilisable comme un résultat complet.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/auth/ApiAuthenticator.kt:55](#)
Modes d'authentification.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/auth/ApiAuthenticator.kt:159](#)
Endpoint token et durée de vie.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/http/RestClientHttpGateway.kt:61](#)
Flux et enveloppe de reprise.

DC-10 Cible non qualifiée

Parcourir les pages API sans perdre le contrat de source

COMPOTEMENT OBSERVÉ

Le résolveur propose absence de pagination, numéros de page, offset, curseur, lien de corps, Link HTTP et OData. SAP OData v2 possède sa stratégie propre ; les autres usages OData passent par l'autre implémentation. L'URL effective provient de la configuration du dataset, du template versionné ou de la

source. Le résolveur de liens accepte aussi des liens absolus. Il faut donc distinguer les protections de certaines découvertes fournisseur du contrôle général des destinations, qui n'est pas complet dans le code lu.

UTILISATEURS ET RÔLES

Collectes HTTP volumineuses, créateurs de connecteurs et opérateurs de reprise.

INTERFACES DANS LE CODE

PaginationConfig ; PaginationStrategyResolver.resolve ; stratégies PageBased, OffsetBased, CursorBased, LinkBased, HeaderLinkBased, OData et ODataV2.

DONNÉES ET ÉTATS

Paramètres de pagination dans ConfigConnector ; pages assemblées en fichier local de collecte.

DÉPENDANCES ET ÉCHANGES

API fournisseur, headers de réponse et curseurs distants.

COMPORTEMENT CIBLE PROPOSÉ

Connectors décrit un checkpoint source/epoch/range lié au run. Chaque run borne pages, octets et temps. Publisher publie l'édition et son checkpoint avant l'acquittement ; un curseur de commande PostgreSQL ne devient jamais le checkpoint publié autoritaire.

MIGRATION DE CETTE CAPACITÉ

Rejouer les configurations de pagination existantes avec fixtures déterministes. Ajouter la détection de boucle, une limite de pages et une politique explicite des liens absolus et changements d'origine.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lien suivant cyclique interrompt la collecte avec une erreur exploitable. Une reprise après publication reconnaît le checkpoint publié ; une interruption avant publication ne fait pas avancer la source.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/pagination/PaginationStrategyResolver.kt:40](#) Types de pagination et URL effective.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/pagination/util/UriResolver.kt:18](#) Acceptation des liens absolus.

DC-11 Cible non qualifiée

Découvrir des API métier et figer leur version

COMPORTEMENT OBSERVÉ

Salesforce, Business Central, Dynamics, SAP et SharePoint ont des versions connues et des templates dédiés. Les routes servent versions, parents, entités et colonnes. Les parents représentent notamment des sociétés ou services selon le fournisseur. La création et certains previews figent la version de lecture. Une incompatibilité fournisseur/configuration typée lève une erreur. Le catalogue seul ne prouve pas l'activation : CreateDataSourceCommandHandler interdit MS_DYNAMICS et SAP_S4HANA. Jira utilise le chemin API générique avec Basic à partir de ses champs ; CLEYROP accepte un token historique ou client_credentials.

UTILISATEURS ET RÔLES

Créateurs de sources SaaS et auteurs de configurations de lecture.

INTERFACES DANS LE CODE

GET /connectors/{connectorType}/versions ; GET /datasources/{id}/parents ; GET /datasources/{id}/entities ; GET /datasources/{id}/entities/{entityName}/columns.

DONNÉES ET ÉTATS

Version API de connexion et de lecture ; noms d'entités, parents et colonnes ; configurations typées propres à chaque fournisseur.

DÉPENDANCES ET ÉCHANGES

Templates versionnés, API de découverte, token endpoints et parseur EDMX.

COMPORTEMENT CIBLE PROPOSÉ

Connectors expose un registre versionné de capacités, avec état implémenté/activé/qualifié et contraintes fournisseur. Le domaine Rust conserve des contrats de lecture distincts de la découverte.

MIGRATION DE CETTE CAPACITÉ

Reprendre chaque configuration avec sa version explicite. Maintenir les connecteurs interdits hors du catalogue utilisable tant qu'un ticket de qualification ne les rouvre. Ne pas transformer le support de DTO en promesse produit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une source Salesforce choisie dans une version reconnue conserve cette version après changement du défaut du registre. Une configuration SAP ne peut pas s'exécuter sur un connecteur d'un autre fournisseur.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceDataController.kt:118` Routes de découverte et de versions.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/ConnectorVersionTemplateRegistry.kt:67` Templates par fournisseur/version.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceCommandHandler.kt:72` Interdiction de création Dynamics/SAP.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/CleyropConnector.kt:75` Mode service account CLEYROP.

DC-12 **Cible non qualifiée****Parcourir et lire des documents SharePoint****COMPORTEMENT OBSERVÉ**

SharePoint utilise OAuth client_credentials et un template Graph v1.0. Le résolveur découvre les drives du site. La stratégie fichier distingue la navigation des dossiers, les métadonnées et le téléchargement binaire ; elle fournit des informations de modification et de validation telles que quickXorHash/cTag. Les nextLink Graph doivent rester sous la racine configurée. Les URL de téléchargement pré-signées passent dans un transport séparé sans en-têtes d'authentification de l'API. Les mêmes documents peuvent servir à une collecte tabulaire ou au contenu de corpus IA.

UTILISATEURS ET RÔLES

Gestionnaires de documents, collecte et ingestion de corpus.

INTERFACES DANS LE CODE

Routes de listing et de contenu ingestion ; SharepointV1FileStrategy ; SharepointV1DriveResolver.

DONNÉES ET ÉTATS

Drives et chemins SharePoint, métadonnées enrichies, génération de listing et fichier téléchargé temporaire.

DÉPENDANCES ET ÉCHANGES

Microsoft Graph, fournisseur OAuth et hôte de téléchargement pré-signé.

COMPORTEMENT CIBLE PROPOSÉ

Connectors fournit un adaptateur SharePoint Rust avec policy Egress distincte pour Graph et les URL de contenu. Files produit un FILESET immuable ou Writer une TABLE selon le parcours ; Embeddings épingle le FILESET publié.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de drives, les chemins et les validateurs. Encapsuler les credentials et qualifier les renommages, suppressions, permissions retirées et téléchargements expirés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un document non modifié évite son transfert ; un document changé pendant le téléchargement porte l'empreinte des octets servis. Une redirection vers une destination non autorisée échoue sans transmettre de token Graph.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/SharepointConnector.kt:36`
Authentification et version Graph.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/sharepoint/v1/SharepointV1DriveResolver.kt:20` Découverte des drives.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/version/sharepoint/v1/SharepointV1FileStrategy.kt:271` Contrôle de racine des nextLink.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/APIConnectorManager.kt:606`
Téléchargement pré-signé sans headers.

DC-13 **Cible non qualifiée****Collecter les fichiers SFTP, dépôt et S3****COMPORTEMENT OBSERVÉ**

La factory instancie des managers fichier dédiés à SFTP/dépôt et S3. SFTP emploie une clé du service et sa configuration de source ; dépôt partage ce transport. S3 porte endpoint, région, bucket, préfixe et credentials. Les managers exposent découverte, lecture, enrichissement des métadonnées et import. Le listing de navigation traite un niveau de dossier et peut résoudre un chemin de fichier. Les descripteurs sont réconciliés pour conserver leurs identifiants et états. La configuration de source garde une règle de sélection des fichiers distincte du format de lecture.

UTILISATEURS ET RÔLES

Producteurs de fichiers, partenaires SFTP et analystes.

INTERFACES DANS LE CODE

POST /datasources/{id}/files ; SftpConnectorManager ; S3ConnectorManager ; FileConnectorManager.

DONNÉES ET ÉTATS

FileDescriptor, chemin relatif, taille, date, statut, checksum ; ressources persistées et cache temporaire distant.

DÉPENDANCES ET ÉCHANGES

Serveurs SFTP, S3 compatible, stockage import interne et clé SSH de service.

COMPORTEMENT CIBLE PROPOSÉ

Connectors implémente la découverte et la lecture bornée ; Files conserve des objets immuables. Une collecte ne supprime ni ne renomme un fichier externe avant que l'édition gouvernée et son checkpoint soient publiés.

MIGRATION DE CETTE CAPACITÉ

Qualifier séparément les chemins, encodages, host keys SSH, S3 compatible et la reprise d'un transfert. Remplacer les connexions en cache propres au processus par des workers fongibles capables de reprendre depuis un journal.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption au milieu d'un fichier ne le marque pas traité. Un fichier externe renommé ou modifié ne réutilise pas un résultat audité appartenant à d'autres octets. Le worker de remplacement termine avec le même identifiant de run.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ConnectorFactoryImpl.kt:189](#) Dispatch des managers fichiers.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/S3Connector.kt:6](#) Paramètres de source S3.

[monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/Connector.kt:195](#) Réconciliation des descripteurs fichiers.

[monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/SftpConnectorManager.kt:274](#) Navigation au niveau de dossier.

DC-14 **Cible non qualifiée****Téléverser un fichier local par morceaux****COMPORTEMENT OBSERVÉ**

Le endpoint multipart reçoit un morceau et les positions Content-Range-Start, End et Max. Le service place les octets dans un répertoire temporaire propre à la source avec nom normalisé. Il exige que la longueur actuelle corresponde au début du morceau et que la taille après ajout soit strictement inférieure à la limite. Le descripteur n'est produit qu'à la fin. Une suppression retire un fichier ou tous les fichiers de la source. Un contrôle asynchrone compare ensuite le fichier au schéma du premier dataset associé et émet un message de conflit, de changement de schéma ou de succès.

UTILISATEURS ET RÔLES

Utilisateurs du frontend important un fichier local.

INTERFACES DANS LE CODE

POST /datasources/{id}/uploadFiles ; DELETE /datasources/{id}/files ; CheckFileAsynchCommandHandler.

DONNÉES ET ÉTATS

Fichier partiel dans le stockage temporaire, puis FileDescriptor persistant ; positions des morceaux fournies par le client.

DÉPENDANCES ET ÉCHANGES

Volume temporaire du pod ; normalisation de nom ; notification Socket.IO via événements de contrôle fichier.

COMPORTEMENT CIBLE PROPOSÉ

Files possède une session d'upload durable avec identifiant, taille attendue, empreintes des morceaux, reprise et finalisation atomique. Connectors/Writer consomment seulement un objet finalisé immuable.

MIGRATION DE CETTE CAPACITÉ

Conserver une façade compatible pour le frontend, mais enregistrer les sessions dans le plan de contrôle et les morceaux dans le stockage objet. Ne plus dépendre du pod initial pour terminer le transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un morceau manquant ou répété avec d'autres octets est refusé sans corruption. Après remplacement du worker, l'upload reprend. Le résultat final porte une empreinte et le contrôle asynchrone correspond à la bonne session.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceLocalFileController.kt:32](#) En-têtes et suppression des uploads.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ChunkedFileUploadServiceImpl.kt:94](#) Append, ordre et limite stricte.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CheckFileAsynchCommandHandler.kt:149](#)

Contrôle du premier dataset associé.

DC-15 **Cible non qualifiée****Paginer une arborescence et mettre en cache sa génération****COMPORTEMENT OBSERVÉ**

La navigation se distingue de l'inventaire utilisé pour la lecture d'un tableur. La requête valide skip non négatif et limit positif. La navigation génère un scope par source, chemin normalisé et filtres. Le cache PostgreSQL stocke une génération ordonnée, un total et des métadonnées enrichies. Le rafraîchissement prend un verrou de scope et remplace la génération ; les pages non initiales réutilisent la génération courante. Une tâche supprime les générations expirées/supplantées avec délai de garde. Les sources LOCAL n'acceptent pas le parcours générique de navigation sans format.

UTILISATEURS ET RÔLES

Explorateurs de fichiers, frontend et ingestion de corpus.

INTERFACES DANS LE CODE

POST /datasources/{id}/files ; RequestFilesDTO ; FolderListingService.serveFromCache/listFromSource/sweep.

DONNÉES ET ÉTATS

FolderListingScope, génération, entrées ordinales, total, expiration et remplacement ; cache distinct des descripteurs métier.

DÉPENDANCES ET ÉCHANGES

PostgreSQL pour cache/verrou et manager du connecteur pour parcourir le dossier.

COMPORTEMENT CIBLE PROPOSÉ

Connectors sert des pages attachées à un identifiant explicite de génération et un budget d'entrées. Les clients peuvent terminer une génération même si une autre a été reconstruite.

MIGRATION DE CETTE CAPACITÉ

Préserver tri, total et enrichissements, puis remplacer skip/limit par un curseur versionné sans ambiguïté. Documenter qu'un listing n'est pas un snapshot transactionnel de la source distante.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le lecteur qui commence une pagination puis observe un rafraîchissement concurrent ne mélange pas deux générations. Un dossier dépassant le budget retourne une erreur définie et ne bloque pas les autres sources.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DatasourceListResourcesQueryHandler.kt:95](#)

Séparation navigation/tableur et validation de type.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/listing/FolderListingService.kt:92](#) Scope, pages et rafraîchissement.

[monorepo · services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/listing/FolderListingService.kt:146](#) Verrou et remplacement de génération.

DC-16 **Cible non qualifiée****Lire les formats et appliquer les préparations d'import****COMPORTEMENT OBSERVÉ**

La factory couvre CSV, tableur XLSX, JSON, XML et Parquet. Les DTO configurent encodage, suffixes/MIME, limite de preview et, selon le format, compression, séparateur CSV, feuille, présence d'en-tête, lignes et colonnes ignorées, trim ou chemin JSON/XML. Les préparations comprennent transposition et ajout de nom de fichier, URL, date ou horodatage d'import. Le domaine expose NONE et GZIP pour la compression. withStandardizeColumns apparaît dans le DTO mais n'est pas converti en préparation dans AdvancedOptionsDTO.toPreparations. Les valeurs par défaut MIME/suffixes de Parquet/XML pointent vers la configuration JSON dans les DTO lus.

UTILISATEURS ET RÔLES

Analystes configurant une collecte et lecteurs de fichiers.

INTERFACES DANS LE CODE

FormatConfigDTO et ses variantes ; GET /datasources/format/default ; AdvancedOptionsDTO ; FormatFactoryImpl.

DONNÉES ET ÉTATS

FileFormat, ConfigConnector, Preparations ordonnées et TargetSchema.

DÉPENDANCES ET ÉCHANGES

Parseurs CSV, XLSX, JSON, XML, Parquet ; cache de fichiers et détection de types.

COMPORTEMENT CIBLE PROPOSÉ

Writer et Connectors partagent des codecs Rust bornés en mémoire et en octets décompressés. La configuration de lecture et les préparations deviennent une révision immuable liée à l'audit du candidat.

MIGRATION DE CETTE CAPACITÉ

Constituer des fixtures par format et option, y compris fichiers malformés et limites de décompression. Décider explicitement du comportement de standardisation et corriger les defaults suspects après mesure de compatibilité.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Chaque fixture produit les mêmes colonnes et valeurs utiles dans preview et import. Un séparateur invalide, une feuille absente, un chemin JSON invalide et une expansion démesurée échouent avec des erreurs stables.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/FormatFactoryImpl.kt:19 Formats réellement instanciés.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/dto/AdvancedOptionsDTO.kt:11 Options et conversion des préparations.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/dto/format/ParquetConfigDTO.kt:17 Defaults de format Parquet.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/fileFormat/FileFormat.kt:15 Compression déclarée.

DC-17 **Cible non qualifiée****Prévisualiser les données et contrôler les colonnes****COMPORTEMENT OBSERVÉ**

Le preview distingue SQL, fichier et API. SQL redécouvre la table, valide une éventuelle customQuery et applique les préparations. Le fichier se résout par chemin de référence, peut imposer un schéma et fournir un arbre JSON. L'API actualise ses descripteurs puis renvoie frame, arbre JSON et chemin de base. Le endpoint headers accepte SQL ou API, avec métadonnées SQL ou arbre JSON. Le contrôle d'en-têtes rejette les doublons sans tenir compte de la casse, les noms trop longs, les types incompatibles et les erreurs de lecture ; il renvoie un RestError en 400.

UTILISATEURS ET RÔLES

Analystes pendant la configuration d'un dataset.

INTERFACES DANS LE CODE

POST /datasources/{id}/datasets/preview ; POST /datasources/{id}/datasets/headers ; POST /datasources/{id}/datasets/check-headers ; POST /datasources/{id}/datasets/sheetNames.

DONNÉES ET ÉTATS

DataFrame, TabularSchema, JsonDatasetHeaders, préparations et descripteurs actualisés lors de certains previews.

DÉPENDANCES ET ÉCHANGES

Managers et parseurs ; source externe ; paramètres de limite d'en-tête et de transposition.

COMPORTEMENT CIBLE PROPOSÉ

Connectors exécute des previews bornés et Writer utilise les mêmes conversions. Datasets conserve le schéma proposé et les écarts, sans confondre un échantillon valide avec la validation du candidat complet.

MIGRATION DE CETTE CAPACITÉ

Préserver les discriminants de réponse SQL/JSON et les noms d'erreurs frontend. Séparer l'inférence d'échantillon de la validation de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un échantillon avec colonnes A et a échoue comme doublon. Un type compatible dans le preview mais invalide plus loin dans le fichier empêche la publication et laisse un rapport lié au candidat.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/PreviewDatasetConfigQueryHandler.kt:75 Trois parcours de preview.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/HeadersDatasetConfigQueryHandler.kt:47 Headers SQL ou API seulement.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/query/CheckTargetSchemaQueryHandler.kt:35 Ordre des contrôles et erreurs.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/controller/DatasetConfigController.kt:113 Traduction HTTP du contrôle d'en-tête.

DC-18 **Cible non qualifiée****Créer une collecte et suivre son exécution distribuée****COMPOTEMENT OBSERVÉ**

La création de collecte arrive par les contrats internes. Elle enregistre d'abord un DatasetCollect avec des opérations WAITING, puis construit la configuration fichier, API ou SQL. Elle infère le schéma si nécessaire et planifie un import asynchrone, sauf en mode PLANNED. Les empreintes initiales persistées sont vidées tant que l'import n'est pas acquis. L'import met l'ingestion en RUNNING, prépare les octets et publie CollectUpdatedEvent pour la suite du traitement. La fin d'exécution reçue depuis process-data déclenche la finalisation et, en échec, la notification de collecte.

UTILISATEURS ET RÔLES

Auteurs de datasets, commandes data-dev, data-model et process-data.

INTERFACES DANS LE CODE

CreateCollectFromDatasourceCommandHandler ; CollectInitCommandHandler ; CollectUpdatedEvent ; ProcessDataEventHandlerKafkaConsumer.

DONNÉES ET ÉTATS

DatasetCollect, ancienne configuration, schéma cible, CollectScheduled et états des opérations ingestion/process.

DÉPENDANCES ET ÉCHANGES

Kafka/Protobuf, data-dev, data-model, process-data, stockage import et notification-center.

COMPOTEMENT CIBLE PROPOSÉ

Flows ordonne un run borné ; Connectors lit, Writer prépare un candidat immuable, Audit le vérifie et Publisher publie. Les événements d'avancement ne remplacent pas le reçu de publication. Les sorties sont une édition Iceberg autoritaire.

MIGRATION DE CETTE CAPACITÉ

Reconstituer la machine d'états et ses transitions avant de déplacer les producteurs. Maintenir un adaptateur événementiel temporaire, puis remplacer la chaîne JVM par des commandes et reçus Rust idempotents.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne entre upload et lancement du traitement laisse un run reprenable, sans fausse réussite. Une panne après publication et avant ack reconnaît la même édition et n'en publie pas une seconde.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:3
65 Enregistrement initial WAITING et préparation.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CreateCollectFromDatasourceCommandHandler.kt:4
08 Empreintes en attente et planification.

monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CollectInitCommandHandler.kt:65 Import/finalisation des opérations.

monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/ProcessDataEventHandlerKafkaConsumer.kt:18
Retour de fin d'exécution.

DC-19 **Cible non qualifiée****Rafraîchir un dataset avec règles de schéma et de doublons****COMPOTEMENT OBSERVÉ**

Le rafraîchissement peut changer fréquence et type APPEND/OVERRIDE. Les fichiers ont contrôles de format, schéma et checksum ; APPEND peut importer les fichiers valides en émettant WARNING si certains sont ignorés, alors que OVERRIDE échoue si un fichier a un schéma invalide. Les API JSON ne passent pas par le même contrôle de checksum. Le SQL relit les colonnes vivantes : STRICT_FIXED détecte colonnes absentes, nouvelles ou de type différent ; PERMISSIVE_FIXED tolère certaines différences et peut autoriser les casts implicites ; MERGE et REPLACE ne produisent pas ces conflits préalables. Une source vide ou déjà traitée possède une issue distincte.

UTILISATEURS ET RÔLES

Utilisateurs du rafraîchissement manuel et déclencheurs de collecte.

INTERFACES DANS LE CODE

POST /datasets/{id}/refresh ; UpdateCollectCommandHandler ; TriggerOperationCommandHandlerUpdate.

DONNÉES ET ÉTATS

RefreshType, SchemaMode, sourceConfig courante et ancienne, checksums et états
WARNING/FAILURE/SKIPPED.

DÉPENDANCES ET ÉCHANGES

Data-model pour les changements de schéma et datasets vidés ; process-data pour matérialisation ; notifications de conflit.

COMPOTEMENT CIBLE PROPOSÉ

Datasets versionne le contrat de schéma ; Writer produit un candidat APPEND ou remplacement sous WAP. Publisher avance la référence publiée seulement après Audit. Les empreintes deviennent des preuves sur octets immuables et les checkpoints publiés décrivent les entrées acquises.

MIGRATION DE CETTE CAPACITÉ

Reprendre les quatre modes de schéma avec une table de compatibilité explicite. Remplacer la notion de fichier traité stockée dans une ancienne config par des participants/checkpoints de publication, en conservant l'historique de provenance.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une collecte APPEND contenant un fichier déjà traité et un fichier neuf n'ajoute que le second. Une incompatibilité STRICT_FIXED bloque la publication ; une évolution MERGE suit la politique de gouvernance et garde l'édition précédente lisible.

Références de l'analyse

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:289](#) Contrôles fichiers et exception du parcours API.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:316](#) Quatre modes de schéma SQL.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/TriggerOperationCommandHandler.kt:418](#) Comportements APPEND et OVERRIDE.

[monorepo · services/data-collect/app/data-collect-](#)

[domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/aggregate/IngestionStatus.kt:24](#) Issues réglées, vide et déjà traité.

DC-20 **Cible non qualifiée**

Planifier les collectes et maîtriser leurs tentatives**COMPOTEMENT OBSERVÉ**

Les fréquences sont MANUAL, PLANNED et RECURRENT ; SchedulerCollect porte cron, fuseau et politique de reprise. Quartz utilise un store JDBC clusterisé et un trigger de retry persistant propre au service. Les opérations ont un UUID différent de celui du dataset, demandent la récupération après panne et exécutent un job non concurrent pour la même clé. Le résultat décide d'arrêter le trigger, de réessayer ou de finaliser à épuisement. Une garde d'admission en mémoire évite pendant cinq secondes deux soumissions du même dataset, sans verrou partagé entre pods. Une nouvelle demande annule les opérations pendantes du dataset.

UTILISATEURS ET RÔLES

Utilisateurs de schedules, déclencheurs et opérateurs.

INTERFACES DANS LE CODE

Événements ScheduleEvent ; SchedulerServiceQuartzImpl ; DatasetScheduledJob ; DatasetServiceAsyncJob ; IngestionRetryTrigger.

DONNÉES ET ÉTATS

CollectScheduled et tables Quartz PostgreSQL ; cron/fuseau/politique ; compteur de tentatives persistant ; garde locale éphémère.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Quartz, data-model et planification Spring au démarrage.

COMPOTEMENT CIBLE PROPOSÉ

Flows conserve en PostgreSQL les schedules et commandes de run, avec lease, deadline, retry budget et fencing applicatif. Les workers restent fongibles. Une occurrence planifiée a une clé idempotente distincte de l'édition publiée.

MIGRATION DE CETTE CAPACITÉ

Exporter les schedules et leurs fuseaux, comparer les prochaines occurrences, drainer les tentatives Quartz puis activer le scheduler Rust sur une frontière de temps. Ne pas compter la garde locale comme garantie de concurrence cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

À deux schedulers concurrents, une seule commande existe pour une occurrence. Un worker mort peut être remplacé sans double publication. Les changements d'heure et les retries conservent le nombre et l'ordre d'occurrences attendus.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/aggregate/DatasetCollectRefreshConfig.kt:10 Fréquences, cron, fuseau et reprise.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/scheduler/service/SchedulerServiceQuartzImpl.kt:119 Opérations et annulation des tentatives pendantes.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/scheduler/service/quartzjob/DatasetServiceAsyncJob.kt:20 Décision de reprise/finalisation.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/dataset/service/InMemoryCollectAdmissionGuard.kt:24 Debounce local au processus.

monorepo · services/data-collect/app/data-collect-app/src/main/resources/application.properties:251 Store Quartz JDBC clusterisé.

DC-21 **Cible non qualifiée**

Préparer les objets d'import et les nettoyer après exécution

COMPOTEMENT OBSERVÉ

Les imports s'écrivent dans un bucket compatible S3 sous un nom d'objet de dataset. Le writer courant supprime le BOM UTF-8 quand il est présent. À la fin de la collecte, le nettoyage parcourt les objets du préfixe et supprime ceux antérieurs au seuil temporel ; il garde ceux déposés après ce seuil pour éviter d'effacer une nouvelle tentative. Les fichiers locaux d'une source peuvent aussi être retirés. Le nettoyage est asynchrone et journalise les échecs sans mécanisme de publication Iceberg dans ce writer.

UTILISATEURS ET RÔLES

Workers de collecte et maintenance des imports temporaires.

INTERFACES DANS LE CODE

S3ObjectWriter.import/deleteAllForDataset ; ConnectorFactoryImpl.deleteImport ; CollectFinishCommandHandler.

DONNÉES ET ÉTATS

Objets temporaires S3 et fichiers locaux ; cutoff stagedBefore et configurations de collecte.

DÉPENDANCES ET ÉCHANGES

Bucket d'import, API S3 et retour process-data.

COMPOTEMENT CIBLE PROPOSÉ

Writer de confiance crée uniquement des fichiers immuables et médie le staging. Publisher médie les commits et la publication. Le worker n'obtient pas table.modify REST, qui inclut aussi drop. Audit lie sa preuve au candidat sans réécrire origin.* par fast-forward. Maintenance supprime sur plan audité avec garde de rétention.

MIGRATION DE CETTE CAPACITÉ

Remplacer les préfixes partagés et cutoffs temporels par des identifiants run/attempt, manifests et reçus. Conserver l'ancien nettoyage pour ses objets legacy seulement jusqu'au drainage.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une ancienne tentative terminée ne peut effacer les fichiers d'une nouvelle. Une preuve d'audit reste liée aux mêmes octets. Un worker ne peut pas commit/drop une table via le catalogue ; seul Publisher peut rendre l'édition visible.

Références de l'analyse

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/repository/S3ObjectWriter.kt:80 Écriture et BOM.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/repository/S3ObjectWriter.kt:121 Suppression par préfixe et ancienneté.

monorepo · services/data-collect/app/data-collect-

domain/src/main/kotlin/com/cleyrop/data/collect/domain/dataset/command/CollectFinishCommandHandler.kt:104 Persistance et déclenchement de nettoyage à la fin.

DC-22 **Cible non qualifiée**

Fournir les fichiers à une ingestion de corpus IA**COMPOTEMENT OBSERVÉ**

Deux routes spécialisées listent les fichiers et téléchargent un fichier au nom d'un utilisateur et d'un projet. Elles appellent un contrôle IAM résiduel avant tout accès aux octets. Le téléchargement exige un connecteur FILE et un manager FileContentFetcher. Il consulte les métadonnées avant de décider un 304 sur If-None-Match. Sinon il télécharge dans un fichier temporaire, calcule quickXor sur les octets et relit les métadonnées si le validateur annoncé ne correspond pas. La réponse expose contenu, taille, MIME, date, ETag et digest selon disponibilité.

UTILISATEURS ET RÔLES

ai-gen-assistant en délégation d'un utilisateur, projets et ingestion de documents.

INTERFACES DANS LE CODE

POST /datasources/{id}/ingestion/files?userId=...&projectId=... ; GET /datasources/{id}/ingestion/files/content avec path et If-None-Match.

DONNÉES ET ÉTATS

Métadonnées de source, validateur conditionnel, fichier temporaire et empreinte réelle des octets servis.

DÉPENDANCES ET ÉCHANGES

IAM on-behalf-of, source S3/SFTP/SharePoint selon capacité du manager, corpus IA consommateur.

COMPOTEMENT CIBLE PROPOSÉ

Connectors et Files servent une délégation bornée à une ressource et un run. Le corpus référence une édition FILESET publiée, puis Embeddings une édition EMBEDDINGS avec provenance vers ses fichiers. Les credentials ne traversent pas vers l'assistant.

MIGRATION DE CETTE CAPACITÉ

Conserver le protocole conditionnel pour réduire les transferts, mais rattacher chaque document et son digest au manifest immuable. Passer les paramètres de délégation dans une autorisation signée vérifiée par IAM.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur retiré du projet ne peut plus lister ni lire les octets par le service IA. Un fichier changé pendant le transfert ne reçoit pas un ETag trompeur ; la provenance des embeddings pointe sur les octets réellement ingérés.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-`

`app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/controller/DataSourceIngestionController.kt:45` Routes de délégation et contrôle préalable.

`monorepo · services/data-collect/app/data-collect-`

`domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/query/DatasourceFileContentQueryHandler.kt:78 304,` téléchargement et vérification des octets.

`monorepo · services/data-collect/app/data-collect-`

`app/src/main/kotlin/com/cleyrop/data/collect/app/common/authz/ResidualAuthzChecks.kt:35` Délégation vers le broker IAM.

DC-23 **Cible non qualifiée**

Administrer les accès de dépôt SFTP et leurs clés

COMPORTEMENT OBSERVÉ

Un accès de dépôt comporte nom de ressource, nom affiché, répertoire, login, propriétaire, statut, expiration et clés SSH. La création vérifie l'unicité login/répertoire, crée le compte par commande SSH distante, ajoute les clés puis persiste. Elle compense les effets partiels tant que la persistance complète n'a pas été atteinte. La modification diffère les clés à ajouter/retirer ; une liste vide n'entre pas dans ce parcours. La suppression retire d'abord le compte distant puis les enregistrements et publie l'événement. La création/suppression de dépôt entraîne ensuite celle de la datasource DEPOT.

UTILISATEURS ET RÔLES

Administrateurs et propriétaires des dépôts de partenaires.

INTERFACES DANS LE CODE

GET/POST /sftp-accesses ; GET/PATCH/DELETE /sftp-accesses/{id} ; GET /sftp-accesses/generate-directory-name ; GET /datasources/sftp/key.

DONNÉES ET ÉTATS

SftpAccess, SftpAccessAuthorization et compte distant ; faits IAM et datasource DEPOT dérivée.

DÉPENDANCES ET ÉCHANGES

Serveur SFTP administré par SSH et commandes système ; base de miroirs utilisateurs ; événements DepotEvent.

COMPORTEMENT CIBLE PROPOSÉ

Files administre les dépôts avec un adaptateur SFTP non JVM. Egress exécute les effets d'administration sous identifiant d'opération et IAM autorise les objets sftp_access. Un reçu permet de réconcilier l'état distant et PostgreSQL.

MIGRATION DE CETTE CAPACITÉ

Reprendre comptes, clés publiques et propriétaires sans les recréer. Remplacer la composition de commandes shell par un protocole d'administration structuré ou des arguments strictement validés. Documenter vider/révoquer toutes les clés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne après création distante puis avant persistance se résout par reprise ou compensation traçable. Retirer toutes les clés a un comportement explicite. Supprimer un accès absent reste idempotent et ne supprime aucun autre compte.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/depot/command/CreateSftpAccessCommandHandler.kt:41` Unicité, effets distants et compensation.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/depot/command/UpdateSftpAccessCommandHandler.kt:40` Diff des clés seulement si liste non vide.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/depot/service/SftpServiceImpl.kt:31` Administration SSH distante.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/DataCollectEventHandlerKafkaConsumer.kt:31` Dépôt vers datasource.

DC-24 Cible non qualifiée

Lister les dépôts et appliquer la rétention des fichiers

COMPORTEMENT OBSERVÉ

Le service liste les accès de dépôt, leurs fichiers et un KPI de total. La liste de fichiers enlève un répertoire protégé et restitue des chemins relatifs. La configuration conserve une durée de rétention commune, modifiable par la route d'administration. Un job cherche récursivement les fichiers plus anciens que le cutoff et les supprime sur le serveur. Le démarrage n'active ce job que si la fonction et un hôte SFTP sont configurés, et annule un ancien job sinon. L'expiration portée par un accès n'établit pas, dans ces handlers, une révocation automatique du compte distant.

UTILISATEURS ET RÔLES

Administrateurs de dépôts et opérateurs de rétention.

INTERFACES DANS LE CODE

GET /sftp-accesses/files ; GET/PATCH /sftp-accesses/configuration ; GET /sftp-accesses/kpi ; CleanDepotJob.

DONNÉES ET ÉTATS

SftpExpireTime global, horodatages distants et jobs de nettoyage ; champ expiredAt d'accès séparé.

DÉPENDANCES ET ÉCHANGES

SFTP/SSH et Quartz ; dossiers protégés configurés.

COMPORTEMENT CIBLE PROPOSÉ

Files et Maintenance appliquent une politique de rétention par tenant et dépôt. Chaque suppression est un effet d'un plan audité qui vérifie leases, rétention et éditions consommatrices. IAM définit le contrat d'expiration d'accès.

MIGRATION DE CETTE CAPACITÉ

Importer la durée actuelle et inventorier les dossiers protégés sans publier leurs valeurs. Remplacer le balayage destructif direct par des candidats de suppression et des reçus. Décider l'effet de expiredAt sur les comptes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un fichier encore nécessaire à une collecte ou retenu par politique n'est pas supprimé. Le plan exclut les dossiers protégés et les chemins hors racine. La reprise d'un plan n'efface pas un fichier nouveau au même nom.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/depot/controller/SftpAccessController.kt:203` Listing, rétention et KPI.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/depot/service/SftpServiceImpl.kt:141](#) Suppression des fichiers anciens hors répertoire protégé.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/configuration/RunAfterStartup.kt:108](#) Activation conditionnelle et annulation du job.

DC-25 Cible non qualifiée

Converger les identités, transférer les propriétés et reconstruire les droits

COMPORTEMENT OBSERVÉ

Les consumers IAM maintiennent des correspondances IAM/ancien UUID de projet et IAM/subject utilisateur. Ils dédupliquent les événements, diffèrent les memberships dont l'utilisateur n'est pas encore mappé, puis les rejouent dans l'ordre lors de la convergence. La suppression désactive le miroir et retire les memberships ; l'offboarding réversible peut les conserver. La réaffectation des sources et datasets est derrière un flag distinct. Le endpoint reconcile-tuples réenonce les faits détenus par data-collect et retourne 202 si activé, 409 sinon. IAM reste le seul écrivain OpenFGA. Malgré la note générale de l'AGENTS, le code conserve des vérifications broker pour ingestion et SFTP pendant la transition.

UTILISATEURS ET RÔLES

IAM, procédures de départ utilisateur et opérateurs de bascule des droits.

INTERFACES DANS LE CODE

IamEventConsumer ; POST /admin/reconcile-tuples ; GET /users/{userId}/check-owner ; ResidualAuthzChecks.

DONNÉES ET ÉTATS

Miroirs User/Project, maps d'UUID, événements différés, processed events, propriétaires et authz facts/outbox.

DÉPENDANCES ET ÉCHANGES

IAM, ancien data-dev/Benutzer, APISIX et OpenFGA via IAM.

COMPORTEMENT CIBLE PROPOSÉ

IAM porte les identités canoniques et les droits. Connectors/Files réenoncent seulement leurs faits métier et appliquent les transferts idempotents. Audit conserve l'ancien et le nouveau propriétaire, le mode de départ et la commande.

MIGRATION DE CETTE CAPACITÉ

Drainer les maps et les memberships différés avant suppression des anciens topics. Comparer l'inventaire des propriétés et la réconciliation des faits ; retirer les doubles contrôles seulement avec le chemin réseau imposé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membership arrivé avant l'identité est appliqué une seule fois après mapping. Un départ réversible n'efface pas les liens à rétablir ; un départ définitif retire les droits et transfère toutes les propriétés autorisées.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/IamEventConsumer.kt:278](#) Membres différés et résolution de clés.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/event/consumer/IamEventConsumer.kt:394](#) Départ utilisateur et transfert sous flag.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/admin/controller/AdminReconcileTuplesController.kt:60](#) 202/409 de réconciliation.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/authz/ResidualAuthzChecks.kt:23](#) Contrôles résiduels réellement présents.

DC-26 **Cible non qualifiée**

Encadrer le réseau sortant et remplacer les dépendances opérationnelles JVM

COMPORTEMENT OBSERVÉ

Le chart restreint l'entrée au gateway et aux pairs déclarés, mais laisse volontairement la sortie réseau non restreinte pour les connecteurs. La validation API contrôle schéma HTTP, ports et syntaxe de chemin/paramètres, sans validation d'IP privée, loopback ou re-résolution DNS dans ce validateur. Le client HTTP utilise des timeouts et masque certains headers ; une politique SSRF de bout en bout ne ressort pas des fichiers lus. Le déploiement utilise probes, PostgreSQL/Flyway, Kafka, volume temporaire optionnel et clés SSH ; le HPA n'est rendu que sans persistance. Ces éléments décrivent les manifests, pas l'état d'un cluster exécuté.

UTILISATEURS ET RÔLES

Opérateurs, sécurité des connecteurs et plateforme d'exécution.

INTERFACES DANS LE CODE

ApiRequestValidator.validateUrl ; RestTemplateConfig ; chart Deployment/NetworkPolicy/HPA ; consumer/producer Kafka.

DONNÉES ET ÉTATS

Configuration, fichiers temporaires, tables Flyway/Quartz et outbox ; connexions et caches en mémoire.

DÉPENDANCES ET ÉCHANGES

JVM/Spring, Kafka JVM actuel, drivers JDBC/Hadoop, PostgreSQL, S3, IAM et services externes.

COMPORTEMENT CIBLE PROPOSÉ

Egress applique destination, port, résolution DNS, redirection, secrets transmissibles, quotas et délai par run. Les cores Rust remplacent Spring/Quartz/JDBC et le bus final exclut Kafka JVM. Les workers n'ont aucune autorité générale de catalogue et doivent obtenir des accès bornés aux octets.

MIGRATION DE CETTE CAPACITÉ

Déployer le contrôle Egress avant ouverture de connecteurs nouveaux. Tester les destinations internes explicitement autorisées et bloquer les autres. Découpler API de contrôle, scheduling et workers de lecture ; migrer l'outbox avec consommateurs compatibles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une URL de métadonnées cloud, un changement DNS vers loopback et une redirection vers un hôte non autorisé sont refusés avant transfert de secrets. Les redémarrages laissent les commandes récupérables et les probes distinguent dépendance indisponible et processus vivant.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/api/request/ApiRequestValidator.kt:81](#) Validation syntaxique, ports et limites sans contrôle d'adresse.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/configuration/RestTemplateConfig.kt:61](#) Configuration du transport HTTP.

[monorepo](#) · [services/data-collect/chart/templates/networkpolicy.yaml:27](#) Absence volontaire de politique egress.

[monorepo](#) · [services/data-collect/chart/templates/hpa.yaml:1](#) Autoscaling conditionné par la persistance.

DC-27 **Cible non qualifiée**

Conserver les événements sortants et isoler les échecs de consommation

COMPORTEMENT OBSERVÉ

Les producteurs métier et de faits IAM déposent leurs payloads dans une outbox PostgreSQL. Le relay prend un lot verrouillé, attend la confirmation Kafka puis renseigne sentAt ; une erreur augmente attempts et conserve la ligne. Les événements historiques sans clé restent sans clé de partition, alors que les faits IAM peuvent être ordonnés par agrégat. Les lignes envoyées sont purgées après une rétention configurable. Les consommateurs rejettent les messages en échec vers une DLT ; le consommateur terminal persiste payload, coordonnées originales et erreur en PostgreSQL, puis laisse l'offset non acquitté si cette persistance échoue. Les enregistrements DLT de data-collect reçoivent un UUID aléatoire à chaque persistance, ce qui ne constitue pas une déduplication du replay.

UTILISATEURS ET RÔLES

Opérateurs, consommateurs IAM/data-dev/process-data et automatisations de reprise.

INTERFACES DANS LE CODE

OutboxPublisher ; OutboxRelay.drain/cleanupSent ; DeadLetterEventConsumer.consume ; KafkaConfiguration.

DONNÉES ET ÉTATS

outbox_event, dead_letter_event et suivi des messages entrants utilisé par certains consumers ; sentAt, attempts et lastError.

DÉPENDANCES ET ÉCHANGES

PostgreSQL et Kafka ; consommateurs de contrats Protobuf et faits IAM.

COMPORTEMENT CIBLE PROPOSÉ

Un relais Rust conserve le même contrat de persistance avant envoi vers un bus sans JVM. Audit garde les coordonnées et causes de rejet. Chaque effet possède une clé d'idempotence stable ; les checkpoints de collecte publiés restent dans l'édition Iceberg et ne se confondent pas avec les offsets de transport.

MIGRATION DE CETTE CAPACITÉ

Inventorier les producteurs et les clés de partition, reprendre l'outbox non envoyée et les quarantaines, puis basculer le transport par consommateur. Fournir une commande de replay audité qui conserve l'identité du message.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper le transport avant et après son acquittement puis redémarrer. Tout événement reste disponible et les doublons éventuels sont absorbés par le consommateur. Un échec PostgreSQL pendant la mise en quarantaine ne fait pas avancer l'offset terminal.

Références de l'analyse

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/outbox/OutboxRelay.kt:23](#) Confirmation d'envoi, reprises et purge des lignes envoyées.

[monorepo](#) · [services/data-collect/app/data-collect-](#)

[app/src/main/kotlin/com/cleyrop/data/collect/app/common/deadletter/DeadLetterEventConsumer.kt:26](#) Persistance terminale et réémission d'erreur.

monorepo · services/data-collect/app/data-collect-

app/src/main/kotlin/com/cleyrop/data/collect/app/common/deadletter/DeadLetterEventConsumer.kt:54 Coordonnées originales et UUID de stockage aléatoire.

Les 19 types de connecteurs déclarés

Les 19 noms de ConnectorType ne constituent pas 19 connecteurs qualifiés. Le remplacement JDBC/JVM se fait par moteur et protocole avec des tests de contrat. Un extracteur OLake Go éventuellement retenu ne couvre pas implicitement ces familles ni leurs authentifications, previews, transformations, versions et règles de refresh. Aucun composant OLake Java ne peut rester dans la cible.

Type / famille	Comportement observé	Cible et écart	Acceptation
LOCAL / Fichiers	Manager local et upload par chunks, descripteurs persistés, formats et préparations.	Files/Connectors Rust : préserver reprise Content-Range, noms normalisés, previews et évolution de schéma.	Upload interrompu/repris puis lecture des mêmes octets ; refus des offsets incohérents et des formats invalides.
SFTP / Fichiers distants	Manager SFTP, navigation/cache et lecture des fichiers avec credential ; distinct des comptes de dépôt administrés.	Adaptateur SFTP sans JVM qualifié pour auth, clés, listing et flux ; Egress et Writer bornent connexions et octets.	Même arborescence filtrée et fichiers sélectionnés ; coupure en lecture puis reprise sans publier un fichier partiel.
DEPOT / Dépôt administré	Manager de fichier partagé avec SFTP ; création/suppression d'accès distant, clés publiques, règles de rétention.	Distinguer gestion des comptes et lecteur de fichiers ; préserver compensation, protections de chemins et revue de la révocation.	Créer, modifier et retirer un accès ; vérifier la révocation réelle, les répertoires protégés et la rétention.
MYSQL / SQL/JDBC	SQLConnector et driver JDBC ; métadonnées, requête restreinte, Parquet d'import.	Lecteur sans JVM à qualifier : types, décimaux, timezone, caractères, vues, cohérence snapshot et streaming.	Extraction d'un corpus MySQL de référence avec égalité des valeurs/types et reprise bornée.
POSTGRESQL / SQL/JDBC	SQLConnector et driver JDBC ; découverte spécifique des objets et import Parquet.	Lecteur Rust ou tiers encapsulé ; conserver types, vues et identifiants, puis définir snapshot/CDC explicitement.	Corpus PostgreSQL incluant types particuliers et vue ; démontrer cohérence de l'édition et reprise sans perte.
ORACLE / SQL/JDBC	SQLConnector et driver JDBC Oracle, métadonnées et lecture SQL.	Choisir un adaptateur autorisé sans JVM ; qualifier connectivité, précision NUMBER, dates, schémas et limites du compte.	Comparer un corpus Oracle et son schéma à l'extraction historique ; timeout et refus de droits restent identifiables.
MSSQL / SQL/JDBC	SQLConnector et driver SQL Server ; schémas/tables et flux vers Parquet.	Lecteur SQL Server sans JVM, avec équivalences de types et stratégie de lecture cohérente.	Valider décimaux, dates, Unicode, noms qualifiés et comportement sous modification concurrente.
BIGQUERY / SQL/JDBC déclaré	Modèle SQL présent mais entrée de driver vide dans JdbcConnectorManager ; exécution non qualifiée.	Construire ou sélectionner un lecteur BigQuery sans JVM, qualifier l'API de jobs/lecture et les coûts ; ne pas annoncer une parité héritée.	Une authentification, découverte, preview et extraction bornée doivent fonctionner dans un environnement de test avant support déclaré.

Type / famille	Comportement observé	Cible et écart	Acceptation
SNOWFLAKE / SQL/JDBC	Modèle SQL et driver JDBC ; requêtes et métadonnées via le chemin commun.	Adaptateur sans JVM à qualifier pour authentification, contexte de schéma et types Snowflake ; contrat de snapshot explicite.	Comparer lignes et types d'un corpus, puis vérifier limites, annulation et reprise d'un export interrompu.
API / HTTP générique	Manager HTTP ; modes d'auth, cache token, pagination, retries, découverte et téléchargement.	Lecteur Rust conservant les stratégies connues, sous politique Egress et budgets pages/octets/temps.	Fixtures de chaque pagination, 401/429, Retry-After, lien absolu et téléchargement coupé.
JIRA / SaaS HTTP	Sous-type API ; endpoint de recherche JQL et Basic utilisateur/token.	Adaptateur Jira sans JVM avec contrat de version, champs et pagination réellement utilisés.	Même recherche et projection sur plusieurs pages ; erreurs d'auth et limites distantes traitées explicitement.
SALESFORCE / SaaS HTTP versionné	Template Salesforce V6 et versions déclarées ; découverte d'entités/colonnes.	Maintenir le pin de version et qualifier OAuth, métadonnées, pagination et types ; évolution versionnée.	Configuration versionnée reproduit entités, schéma et résultat après changement de version par défaut.
MS_DYNAMICS / SaaS déclaré mais bloqué en création	Modèle/template API v9.2 présents ; CreateDataSourceCommand Handler interdit ce type.	Décider maintien de configurations historiques ou activation future ; qualifier entièrement avant ouverture.	Une matrice de tests de connexion, découverte, preview et collecte valide le choix produit avant support annoncé.
SAP_S4HANA / SaaS déclaré mais bloqué en création	Modèle et templates OData v2/v4 ; création interdite par le handler.	Qualifier les deux variantes OData et le schéma SAP retenu ; ne pas assimiler présence de template et disponibilité produit.	Corpus OData v2 et v4 avec pagination et erreurs ; création reste interdite jusqu'à validation explicite.
BUSINESS_CENTRAL / SaaS HTTP versionné	Template API v2.0, contexte tenant/environnement et authentification token.	Adaptateur sans JVM conservant contexte, sélection d'entités, version et pagination.	Deux environnements ne mélangent ni tokens ni entités ; résultat multi-page identique à la référence.
GOOGLESHEET / AirByte déclaré incomplet	GoogleSheetConnector dérive d'AirByteConnector, sans manager dans la factory ; TriggerOperation contient NotImplementedError pour AIRBYTE.	Construire un adaptateur Sheets sans JVM ou retirer explicitement le type ; aucune couverture implicite par OLake.	Avant activation, démontrer auth, sélection de feuille, entêtes, types, preview et collecte ; sinon réponse unsupported explicite.
S3 / Objet cloud	Manager S3 avec cache distant, navigation et import ; credentials et configuration de bucket côté connexion.	Adaptateur S3 Rust séparant bucket externe, staging Writer et stockage des éditions ; préserver sélection et formats.	Lister/importer des clés avec pagination et noms particuliers ; tester changement d'objet en cours de lecture et accès refusé.
CLEYROP / API de plateforme	Sous-type API ; token historique ou OAuth client_credentials selon configuration.	Conserver un client de plateforme versionné, références d'édition et mandat inter-tenant explicite ; retirer les credentials permanents des workers.	Importer sous l'identité autorisée sans élargissement de droits et vérifier le pin de l'édition lue.

Type / famille	Comportement observé	Cible et écart	Acceptation
SHAREPOINT / Documents cloud HTTP	Template Graph v1.0, découverte de drives, dossiers/fichiers, métadonnées, ETag et téléchargement pré-signé.	Adaptateur Graph sans JVM : qualifier drives, versionnement, liens de téléchargement, rotation des tokens et Egress à chaque destination.	Pagination de dossiers, 304, fichier modifié entre métadonnées et téléchargement et URL de téléchargement refusée hors politique.

Références de l'analyse

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/aggregate/connector/Connector.kt:35` Liste des 19 types.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/ConnectorFactoryImpl.kt:189` Branches effectivement câblées et cas unsupported.

`monorepo · services/data-collect/app/data-collect-domain/src/main/kotlin/com/cleyrop/data/collect/domain/datasource/command/CreateDataSourceCommandHandler.kt:72` Types interdits en création.

`monorepo · services/data-collect/app/data-collect-app/src/main/kotlin/com/cleyrop/data/collect/app/datasource/service/manager/connector/JdbcConnectorManager.kt:736` Drivers JDBC dont BIGQUERY vide.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Connectors possède catalogue des types réellement qualifiés, définitions de connexion, versions, références de secrets, droits d'usage et workflows de demande.
- Flows gère schedules, occurrences, commandes idempotentes, tentatives bornées, leases et échéances ; PostgreSQL reste autoritaire pour ces commandes et les drafts.
- Datasets porte schéma et contrat d'édition ; la référence Iceberg publiée et ses origin.* constituent la preuve autoritaire de données. Les checkpoints publiés ne résident pas uniquement dans PostgreSQL.
- IAM et Audit enregistrent délégations, propriété et décisions. Une identité de worker ne possède jamais un droit générique table.modify au catalogue.

Plan de données proposé

- Lecteurs SQL, SFTP, S3 et HTTP produisent des batches ou fichiers immuables avec compteurs, limites et provenance. Chaque adaptateur déclare sa stratégie de snapshot et de checkpoint.
- Writer médie le staging de confiance et remet un candidat immuable à Audit. Publisher médie publication et ack source après commit publié. Tous les kinds, y compris STREAM et la compaction, suivent WAP.
- Un flux éphémère doit être journalisé durablement avant sa gouvernance. La reprise s'appuie sur source/epoch/range/participants. Une release multi-sorties est elle-même une édition Iceberg et ses lecteurs restent épinglés.
- Files expose FILESET et contenu de corpus ; Maintenance réalise GC/expiration par plan audité et garde de rétention. EXPORT reste un effet et EXPOSURE un consommateur déclaré.

Découpage du core Rust

- Proposition : modules `connector_registry`, `credential_ref`, `connection_check`, `source_discovery`, `source_reader` et `pagination_state`, chacun avec contrats de version et d'erreur.
- Proposition : codecs tabulaires communs au preview et au Writer, adaptateurs par moteur SQL sans JDBC, adaptateurs HTTP/S3/SFTP et lecteur SharePoint avec métadonnées conditionnelles.
- Proposition : scheduler et run coordinator Rust sur PostgreSQL pour commandes, avec fencing applicatif indépendant des seuls CAS UUID/refs/schema/spec et de l'ascendance Iceberg.
- Proposition : interfaces `trusted_writer.stage`, `auditor.attest`, `publisher.publish` et `maintenance.plan`. Elles lient le reçu de publication, les checkpoints et les octets audités.

Identité, autorisation et isolation

- L'AGENTS dit que l'autorisation est déléguée au gateway ; le code montre encore des brokers résiduels pour ingestion et SFTP. Reprendre les règles effectivement exécutées et leur flag de bascule.
- Le périmètre réseau entrant est une condition de confiance de X-Userinfo. L'egress actuel reste ouvert ; valider URL syntaxiquement ne constitue pas une défense SSRF complète.
- Masquer les secrets dans DTO, audit et logs ; porter les secrets par référence et réduire leur durée d'accès. Le mécanisme de conservation sur champ vide mérite un contrat explicite.
- Séparer tenant, source, fichier, run et édition. Valider les chemins relatifs, les host keys SSH, les autorisations on-behalf-of et les URL pré-signées à chaque frontière.
- Les fichiers audités ne doivent jamais être réécrits ; un transfert ou une suppression a une commande et un reçu. Les contrôles de schéma ne remplacent pas les contrôles d'accès aux octets.

Exploitation et reprise

- Le chart décrit un Deployment non root avec ports applicatif et management, probes liveness/readiness, configuration et secrets injectés, certificats et volume temporaire optionnel. Aucun état déployé n'a été inspecté.
- Quartz persiste jobs/triggers dans PostgreSQL et reconstruit les schedules au démarrage. Les caches de connexion et le debounce d'admission restent dans le processus.
- Le HPA est conditionné à l'absence de persistance. Le stockage temporaire et les sessions d'upload imposent de vérifier la topologie réelle avant toute promesse de scale horizontal.
- Flyway accepte des migrations out-of-order et le framework de migrations de code peut arrêter le processus avec code 0. La cible doit rendre les migrations verrouillées, reprenables et leur échec visible.
- Prévoir sauvegarde cohérente des définitions, maps IAM, schedules et outbox, ainsi que des clés requises pour déchiffrer l'historique. Les caches restent reconstructibles ; les objets de données suivent leur politique de rétention.
- Aucune qualification runtime n'est revendiquée. Les tests identifiés couvrent notamment uploads ordonnés, taille, isolation par source et téléchargements conditionnels d'ingestion.

Séquence de migration

- 1. Figer l'inventaire par type de connecteur, configuration réellement utilisée, source/projet/propriétaire et version API ; qualifier les types déclarés mais inactifs.
- 2. Importer UUID, historique masqué, permissions, schedules et références de secrets dans le contrôle Rust ; convertir les identifiants legacy avec preuves de correspondance.
- 3. Qualifier codecs et lecteurs sur fixtures puis sources de test ; comparer previews, schémas, métadonnées et octets, sans doubler les effets de collecte sur les sources.
- 4. Introduire Writer/Audit/Publisher et le journal durable d'entrées ; déplacer les checkpoints vers les éditions publiées et conserver un historique séparé des publications approuvées.
- 5. Basculer les schedules à une frontière d'occurrence, drainer Quartz et les événements existants, puis migrer les comptes SFTP et uploads encore actifs.
- 6. Retirer Spring, Quartz, drivers JDBC/Hadoop et Kafka JVM seulement après drainage et preuve que le plan de contrôle, les workers et les outils d'exploitation n'en dépendent plus.

Écarts et décisions encore ouverts

- Le code expose 19 types mais n'en qualifie pas 19 : GoogleSheet n'a pas de manager Airbyte ; BigQuery a un driver vide ; Dynamics et SAP sont interdits en création. Quelle compatibilité produit est requise pour chacun ?
- UpdateDataSourceCommandHandler.checkDepotDisplayName retourne true quand un dépôt de ce nom existe, alors que son appel et son log suggèrent un contrôle d'unicité. Le comportement réel doit être caractérisé avant correction.
- withStandardizeColumns n'apparaît pas dans la conversion des préparations. ParquetConfigDTO et XMLConfigDTO prennent des defaults JSON. Décider ce qui est un défaut historique à corriger et ce qui doit rester compatible.
- Le debounce d'admission local n'assure pas un verrou par dataset entre plusieurs pods. Définir la politique cible pour rafraîchissements concurrents, annulation et remplacement.
- Les protections de destination doivent couvrir JDBC, SFTP, S3, token endpoints, pagination et téléchargements pré-signés. La lecture statique n'établit pas une protection globale contre SSRF ou DNS rebinding.
- Le statut et expiredAt d'un accès SFTP ne prouvent pas sa révocation distante automatique. Quel comportement est attendu pour désactivation, expiration et suppression de toutes les clés ?
- Identifier les garanties de snapshot des sources SQL et API et l'écart entre comptage séparé et flux lu. Une simple comparaison de nombre de lignes ne suffit pas à prouver l'édition source.
- Le schéma Flyway de data-collect n'entre pas dans le linter Alembic du monorepo. Planifier Expand/Migrate/Contract et rollback sans retirer la compatibilité de lecture au même tag.

Références de l'analyse

- [monorepo](#) · [services/data-collect/AGENTS.md:1](#) Rôle, organisation et obligations opérationnelles du composant.
- [monorepo](#) · [services/data-collect/app/gradle/libs.versions.toml:22](#) Version Kotlin ; Spring Boot déclaré plus bas.
- [monorepo](#) · [services/data-collect/chart/templates/deployment.yaml:50](#) Conteneur, probes, secrets et volumes.
- [monorepo](#) · [services/data-collect/chart/templates/networkpolicy.yaml:23](#) Réseau entrant et absence d'egress restrictif.